

# Evidence-Backed Release Assurance for Autonomous Agent Deployments: Cryptographic Attestation, Fail-Closed Gates, and Tamper-Resilient Audit Infrastructure

Anonymous Submission  
*Submission ID: EVI-227*

## Abstract

As autonomous agentic AI systems assume critical operational responsibilities—executing software workflows, interacting with financial APIs, and modifying persistent databases—releasing new model checkpoints or expanding tool privilege scopes introduces severe security risks. Traditional CI/CD deployment pipelines rely on informal, self-reported status labels or unauthenticated build logs before authorizing releases. This paper presents Evidence Assurance (EviAssure), a cryptographic release assurance framework engineered specifically for autonomous agent deployments. EviAssure introduces: (1) an asymmetric Ed25519 and SHA-256 Merkle tree execution trace attestation engine that seals agent action histories into immutable evidence packs; (2) an in-toto v0.2 / SLSA v1.0 compatible `EvidenceBundle` specification with nonces, timestamps, multi-signature threshold gating, and privacy-preserving salted trace parameter blinding; (3) a fail-closed `ReleasePolicyEngine` enforcing multi-factor release conditions, KMS key ARN verification, and key revocation list (CRL) validation; and (4) an adversarial release tamper evaluation harness comprising 12 distinct attack vectors. Empirical benchmarks demonstrate that EviAssure achieves a 100.0% fail-closed block rate across all 12 attack vectors (compared to 0.0% for standard CI gates and 25.0% for OPA/Sigstore) while processing policy gate evaluations at up to **5,919 operations/second** across 8 parallel cores, scaling Merkle tree construction for  $N = 100,000$  execution traces in 38.1 ms (a packaging overhead of **0.116%**) with  $O(\log N)$  sparse proof compression.

## 1 Introduction

Autonomous software agents driven by Large Language Models (LLMs) operate dynamically across complex tool environments, API endpoints, and persistent memory stores [7, 13, 29, 30]. Unlike static software artifacts whose behaviors are deterministically governed by compiled code, autonomous agents exhibit non-deterministic reasoning patterns, dynamic

tool execution flows, and adaptive multi-step decision sequences. When deploying agent updates, expanding tool privilege scopes, or promoting agentic pipelines to production, organization-wide security depends upon verifying that safety invariants, authorization constraints, and regression test suites have been rigorously validated [7, 20, 31].

Existing software delivery and release pipelines suffer from a fundamental trust defect: traditional deployment gates evaluate unauthenticated status signals (such as process exit codes, plain text console logs, or un-signed JSON build status files) generated by intermediate runner environments. An attacker who gains control over an intermediate build step, manipulates agent memory traces, or exploits prompt injection vectors can easily forge release approval signals, bypassing security verification entirely [5, 18, 24, 27].

To address these critical vulnerabilities, EviAssure introduces an evidence-backed release assurance architecture that replaces informal release signals with cryptographically authenticated Evidence Bundles backed by Merkle inclusion proofs, timestamp freshness windows, Ed25519 digital signatures, KMS key ARNs, and SLSA v1.0 provenance envelopes.

## 1.1 Key Contributions

The primary technical and systems contributions of this work include:

1. **Cryptographic Trace Attestation Engine:** Formalization of an Ed25519 public-key signature scheme [2, 4] coupled with a SHA-256 Merkle tree aggregation protocol that condenses up to  $N = 1,000,000$  agent execution traces into a single immutable 64-byte root digest.
2. **Multi-Signature Threshold Gating:** Support for multi-signature attestation pipelines, enabling threshold verification policies (e.g., requiring valid signatures from at least  $K$  out of  $M$  authorized keys) before authorizing production deployments.
3. **Salted Trace Parameter Blinding:** Introduction of a salted HMAC parameter blinding mode

(blind\_payload) using keyed hashes [9] to conceal sensitive prompt text, proprietary API inputs, and customer PII while preserving public Merkle inclusion auditability.

4. **Browser DOM Visual Attestation:** Extension of execution trace attestation to capture browser UI actions, canonical DOM state hashes ( $\text{SHA256}(\text{DOM}_{\text{canonical}})$ ), and visual screenshot render digests ( $\text{SHA256}(\text{PNG}_{\text{render}})$ ).
5. **Supply Chain Envelope Compatibility:** Provision of full envelope formatting compatible with in-toto v0.2 and SLSA Provenance v1.0 specification standards [24, 26, 27].
6. **Fail-Closed Zero-Trust Policy Engine:** Implementation of a zero-trust policy engine (ReleasePolicyEngine) that evaluates signed evidence packs against declarative governance specifications under a mandatory fail-closed default with Key Revocation List (CRL) and Hardware Security Module (KMS) key ARN enforcement [8, 11, 14].
7. **Game-Based Formal Security Proofs:** Sequence-of-games security proofs establishing reduction to EUF-CMA in the Random Oracle Model [1] and hash collision resistance.
8. **Corpus Forensic Infrastructure:** Delivery of a 1,050-profile multi-architecture agent trace corpus (`corpus/agent_trace_corpus.json`) with 50 adversarial anomaly variants and a CLI forensic inspection tool (`scripts/audit_trace_forensics.py`).
9. **Empirical Evaluation:** Comparative evaluation demonstrating a 100.0% block rate for EviAssure versus 0.0% for standard CI and 25.0% for OPA/Sigstore across 12 adversarial tamper vectors.

## 2 System Architecture and Threat Model

This section outlines the architectural foundation of EviAssure, defining its cryptographic boundary, formal security definitions, and threat model for autonomous agent release pipelines.

### 2.1 Hardware Enclave & KMS Boundary

To defend against local runner compromise, EviAssure roots signing authority within a Hardware Security Module (HSM) or Cloud KMS boundary (`kms://aws/arn:aws:kms:us-east-1:000000000000:key/usenix-release-gate`). The runner process requests attestation signatures over canonical evidence digests without ever exposing private key material to the execution environment.

### 2.2 Formal Security Definitions

**Definition 1** (Unforgeability under Chosen Message Attack (EUF-CMA)). *An Evidence Assurance Scheme  $\Sigma = (\text{KeyGen}, \text{Package}, \text{Verify})$  is EUF-CMA secure if for all probabilistic polynomial-time adversaries  $\mathcal{A}$ , the probability that  $\mathcal{A}$  outputs a valid pair  $(E^*, \sigma^*)$  such that  $\text{Verify}(\text{PK}, E^*, \sigma^*) = 1$  without  $E^*$  having been signed by an honest party holding  $\text{SK}$  is negligible in security parameter  $\lambda$ :*

$$\Pr[\text{Exp}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1] \leq \text{negl}(\lambda) \quad (1)$$

**Definition 2** (Merkle Trace Inclusion Soundness). *For root  $R_M$  and leaf  $H_i$ , a Merkle proof  $\pi_i$  is sound if no computationally bounded adversary can construct  $\pi'_i$  for  $H'_i \neq H_i$  such that  $\text{VerifyMerkle}(H'_i, \pi'_i, R_M) = 1$ .*

**Definition 3** (Salted Trace Parameter Blinding Indistinguishability). *A blinded trace record  $T_{\text{blinded}}$  produced with uniform salt  $S \leftarrow \{0, 1\}^\lambda$  satisfies computational indistinguishability: no PPT adversary  $\mathcal{A}$  can distinguish  $T_{\text{blinded}}$  from a uniformly random bitstring of equal length without knowledge of  $S$ .*

### 2.3 Threat Model and Adversary Capabilities

The security model considers a powerful adversary  $\mathcal{A}$  attempting to deploy an unapproved, regression-prone, or malicious agent update to production environments [6, 19, 21]. The adversary  $\mathcal{A}$  may control intermediate runner environments, network transit layers, or build scripts. We categorize  $\mathcal{A}$ 's capabilities into five distinct attack vectors:

- $\mathcal{A}_1$  (**Runner & Memory Trace Tampering**): Modifying in-memory agent execution logs, altering action output hashes, or dropping failed execution steps to present a clean trace history. In an agentic setting, an attacker attempting to deploy a compromised model update might suppress execution traces recording unauthorized system shell commands or SQL table drop queries.
- $\mathcal{A}_2$  (**Network & Signature Forgery**): Injecting forged cryptographic signatures, stripping signature fields, or signing evidence payloads with unauthorized key pairs. An adversary controlling a CI builder might strip Ed25519 signatures or sign payloads using arbitrary self-generated keys.
- $\mathcal{A}_3$  (**Trace Replay & Time Skew**): Replaying historical, previously approved evidence bundles from earlier release cycles or injecting post-dated timestamps into the future to bypass validity windows. An attacker who cannot forge signatures might replay an old, benign evidence pack from a previous release to authorize a malicious deployment.
- $\mathcal{A}_4$  (**Key Revocation & Governance Bypass**): Using revoked key IDs listed in the Key Revocation List (CRL)

or attempting KMS ARN parameter confusion. If an organization revokes a key following developer credential leaks,  $\mathcal{A}$  attempts to use the revoked key before CRL updates propagate.

- $\mathcal{A}_5$  (**Serialization & Key Malleability Injection**): Injecting uncanonicalized JSON key fields or duplicate dictionary keys to exploit parser discrepancies between packager and verifier. By injecting duplicate JSON dictionary keys or non-standard white-spacing,  $\mathcal{A}$  attempts to cause signature verification to succeed over one view while policy parsing evaluates another.

The release gate enforces a strict zero-trust fail-closed guarantee: any evidence payload matching any of these attack vectors is rejected with exit code 1 (BLOCKED).

### 3 Cryptographic Evidence Attestation

EviAssure establishes verifiable release claims by packaging agent action histories into cryptographically attested evidence bundles. This section describes the Merkle tree aggregation protocol, browser DOM attestation, trace parameter blinding, and SLSA v1.0 / in-toto predicate envelopes.

#### 3.1 Execution Trace Browser UI Attestation Schema

Each execution trace record  $T_i$  captures structured action metadata across autonomous agent steps. For API and backend tool calls,  $T_i$  records:

$$T_i = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{status}_i, \text{duration}_i, H_{\text{out},i}) \quad (2)$$

For browser and UI agent steps, EviAssure extends attestation to capture DOM state hashes ( $H_{\text{DOM}}$ ) and visual screenshot render digests ( $H_{\text{screen}}$ ):

$$T_{\text{UI},i} = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{url}_i, \text{elem}_i, H_{\text{DOM},i}, H_{\text{screen},i}) \quad (3)$$

Each record produces a canonical leaf digest  $H_i = \text{SHA256}(T_i.\text{to\_hash}())$ .

#### 3.2 Salted Trace Parameter Blinding Protocol

To protect proprietary prompt text, customer PII, or sensitive API parameters during public audit, EviAssure implements a keyed HMAC blinding mode (`blind_payload`). Given raw output payload  $P_i$  and salt  $S \leftarrow \{0, 1\}^{256}$ :

$$H_{\text{blinded},i} = \text{BLINDED-} \parallel \text{HMAC-SHA256}(S, P_i)[16] \quad (4)$$

The blinded hash replaces raw payload strings in leaf construction while preserving global Merkle tree inclusion verification.

#### 3.3 Merkle Trace Aggregation and Sparse Proof Compression

To attest  $N$  execution traces without transmitting full raw logs to every gate request, EviAssure aggregates normalized trace digests into a SHA-256 Merkle tree [12, 15]. Each trace record  $T_i$  maps to leaf digest  $H_i$ .

For  $N = 1,000$  traces, transmitting raw trace logs requires 222.6 KB of storage, scaling to 220,705.03 KB for  $N = 1,000,000$  traces. EviAssure provides sparse Merkle proof compression, transmitting the 64-byte root  $R_M$  and  $O(\log N)$  inclusion proof paths  $\pi_i$  for audited steps.

Given proof path  $\pi_i = \{(P_1, \text{pos}_1), \dots, (P_K, \text{pos}_K)\}$ , inclusion is verified recursively by setting  $C_0 = \text{SHA256}(H_i)$  and evaluating:

$$C_j = \begin{cases} \text{SHA256}(C_{j-1} \parallel P_j), & \text{if pos}_j = \text{right} \\ \text{SHA256}(P_j \parallel C_{j-1}), & \text{if pos}_j = \text{left} \end{cases} \quad (5)$$

The proof is valid if and only if  $C_K = R_M$ , reducing transmission size to  $< 5$  KB.

```
def build_merkle_tree(traces):
    leaves = [t.to_hash() for t in traces]
    current = leaves
    levels = [leaves]
    while len(current) > 1:
        next_level = []
        for i in range(0, len(current), 2):
            left = current[i]
            right = current[i+1] if i+1 < len(current)
            else current[i]
            next_level.append(hash_sha256(f"{left}:{right}"))
        current = next_level
        levels.append(current)
    return current[0], levels
```

Listing 1: Merkle Trace Aggregation Protocol.

#### 3.4 SLSA v1.0 and in-toto Predicate Envelopes

To ensure seamless integration with modern cloud-native artifact registries and software supply chain attestors, EviAssure formats each generated EvidenceBundle into an in-toto v0.2 Statement envelope compatible with SLSA Provenance v1.0 specifications [24, 27]. Listing 2 illustrates the JSON structure exported by EviAssure.

The envelope schema binds the target release artifact (e.g., agent model weight package, code container, or configuration tarball) to the evidence attestation. The subject field specifies the artifact filename and its SHA-256 digest. The predicateType URI asserts SLSA Provenance v1.0 compliance. Within predicate.buildDefinition, the buildType asserts policy gate evaluation parameters, while externalParameters anchors the unique evidence\_id, binding the external evidence pack directly to the SLSA build statement.

```
{
  "_type": "https://in-toto.io/Statement/v0.1",
```

```

"subject": [
  {
    "name": "agentic_release_artifact.tar.gz",
    "digest": { "sha256": "03126da4143384d8..." }
  }
],
"predicateType": "https://slsa.dev/provenance/v1",
"predicate": {
  "buildDefinition": {
    "buildType": "https://usenix.org/
    AgenticReleasePolicy/v1",
    "externalParameters": { "evidence_id": "EVD-0
    b6b6567" }
  }
}
}

```

Listing 2: SLSA Provenance v1.0 Statement envelope produced by EviAssure.

## 4 Fail-Closed Policy Verification Engine

Release authorization in EviAssure is governed by a zero-trust policy engine (ReleasePolicyEngine) that enforces mandatory fail-closed default rules. This section details the declarative policy schema, multi-factor evaluation protocol, key governance rules, and formal game-based security proof.

### 4.1 Declarative Policy Governance Rules

A release policy specification  $P$  defines strict quantitative safety thresholds and cryptographic verification conditions:

$$P = (P_{\min\_pass}, P_{\max\_drift}, P_{\min\_sigs}, P_{\text{valid\_window}}, \mathcal{K}_{\text{rev}}, \mathcal{A}_{\text{KMS}}) \quad (6)$$

where:

- $P_{\min\_pass}$ : Minimum required unit and integration test pass rate (default: 100.0% (100/100)).
- $P_{\max\_drift}$ : Maximum allowable unresolved security drift findings (default: 0).
- $P_{\min\_sigs}$ : Required threshold number of valid signatures from authorized keys (default:  $K = 1$ ).
- $P_{\text{valid\_window}}$ : Evidence timestamp freshness validity window  $\Delta T_{\max} = 3,600$  s (1 hour) with a maximum future clock skew tolerance of  $\delta_{\text{skew}} = 30$  s.
- $\mathcal{K}_{\text{rev}}$ : Key Revocation List (CRL) tracking compromised or deprecated key identifiers.
- $\mathcal{A}_{\text{KMS}}$ : Required Cloud KMS or HSM key ARN boundary pattern matching.

Listing 3 outlines the complete zero-trust verification algorithm enforced by ReleasePolicyEngine.

```

# Protocol 2: Fail-Closed Policy Gate Verification
# Input: Evidence E, Policy P, Revoked keys K_rev, Seen
#         nonces N_seen
# Output: Decision D in {APPROVED, BLOCKED}

```

```

def evaluate_release_gate(E, P, K_rev, N_seen):
    if not E.signed or E.signature is None:
        return BLOCKED, ["POLICY_VIOLATION:_Unsigned_
        payload"]

    # 1. Signature & Key Governance Verification
    valid_sigs = 0
    for s_entry in E.signatures:
        if s_entry.key_id in K_rev:
            continue
        if s_entry.kms_arn and not match_kms_arn(s_entry.
            kms_arn, P.kms_pattern):
            continue
        if verify_signature(s_entry):
            valid_sigs += 1

    if valid_sigs < P.min_signatures:
        return BLOCKED, ["POLICY_VIOLATION:_Insufficient_
        signatures"]

    # 2. Merkle Root Integrity Verification
    recalculated_root, _ = build_merkle_tree(E.traces)
    if recalculated_root != E.merkle_root:
        return BLOCKED, ["POLICY_VIOLATION:_Merkle_root_
        mismatch"]

    # 3. Quality & Security Drift Thresholds
    if E.test_pass_pct < P.min_pass_pct:
        return BLOCKED, ["POLICY_VIOLATION:_Sub-threshold
        _pass_rate"]
    if E.unresolved_drift > P.max_drift:
        return BLOCKED, ["POLICY_VIOLATION:_Unresolved_
        drift"]

    # 4. Nonce Replay & Freshness Windows
    if E.nonce in N_seen:
        return BLOCKED, ["POLICY_VIOLATION:_Replayed_
        nonce"]
    if not is_timestamp_fresh(E.timestamp, P.valid_window
        , max_skew=30):
        return BLOCKED, ["POLICY_VIOLATION:_Timestamp_
        expired/skewed"]

    N_seen.add(E.nonce)
    return APPROVED, []

```

Listing 3: Fail-Closed Policy Gate Verification Protocol.

### 4.2 Game-Based Formal Security Proof

**Theorem 1** (Fail-Closed Release Soundness). *Under the assumption that Ed25519 signatures are EUF-CMA secure and SHA-256 is collision-resistant, no probabilistic polynomial-time adversary  $\mathcal{A}$  can induce ReleasePolicyEngine to output APPROVED for a modified trace history, forged signature, replayed nonce, or non-compliant evidence payload except with negligible probability  $\text{negl}(\lambda)$ .*

*Proof.* The proof structures the security argument using a formal sequence of games  $\text{Game}_0 \rightarrow \text{Game}_1 \rightarrow \text{Game}_2 \rightarrow \text{Game}_3 \rightarrow \text{Game}_4$ :

- **Game<sub>0</sub>**: Standard execution of adversary  $\mathcal{A}$  interacting with an honest evidence packager and policy gate verifier.
- **Game<sub>1</sub>**: Identical to Game<sub>0</sub>, except the verifier aborts if  $\mathcal{A}$  produces a valid signature  $\sigma^*$  over a forged payload  $E^*$  under key  $PK$  without  $SK$ . The probability of



$\mathcal{A}$  distinguishing Game<sub>0</sub> from Game<sub>1</sub> is bounded by the EUF-CMA security of Ed25519:

$$|\Pr[G_0] - \Pr[G_1]| \leq \text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A}) \quad (7)$$

- **Game<sub>2</sub>**: Identical to Game<sub>1</sub>, except the verifier aborts if  $\mathcal{A}$  produces a trace set  $\mathcal{T}^* \neq \mathcal{T}$  such that  $\text{MerkleRoot}(\mathcal{T}^*) = R_M$ . The difference between Game<sub>1</sub> and Game<sub>2</sub> is bounded by the collision resistance of SHA-256:

$$|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}_{\text{SHA256}}^{\text{coll}}(\mathcal{A}) \quad (8)$$

- **Game<sub>3</sub>**: Identical to Game<sub>2</sub>, except the verifier aborts if  $\mathcal{A}$  submits a replayed evidence nonce  $N^* \in N_{\text{seen}}$ . Since nonces are recorded in the state memory  $N_{\text{seen}}$ , the probability of reuse acceptance in Game<sub>3</sub> is identically zero.
- **Game<sub>4</sub>**: Identical to Game<sub>3</sub>, except the verifier aborts if  $\mathcal{A}$  uses a revoked key  $K^* \in \mathcal{K}_{\text{rev}}$  or an out-of-bounds KMS ARN. Rejection in Game<sub>4</sub> is deterministic.

Combining the game hops, the total advantage of adversary  $\mathcal{A}$  is strictly bounded:

$$\begin{aligned} \text{Adv}_{\text{EviAssure}}^{\text{tamper}}(\mathcal{A}) &\leq \text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A}) + \text{Adv}_{\text{SHA256}}^{\text{coll}}(\mathcal{A}) \\ &\leq \text{negl}(\lambda) \end{aligned} \quad (9)$$

This completes the proof.  $\square$

## 5 Real-World Case Study and Production CI/CD Integration

To evaluate operational deployment feasibility, EviAssure was integrated into a production CI/CD workflow. This section presents the GitHub Action architecture, browser UI attestation runner, and the 1,050-profile agent execution trace corpus.

EviAssure was deployed as a production GitHub Action ([.github/actions/evidence-release-gate/action.yml](https://github.com/actions/evidence-release-gate/action.yml)) and an in-toto v0.2 layout specification ([governance/intoto\\_layout.json](https://github.com/governance/intoto_layout.json)). Evaluation tested two primary agentic workflow specimens:

1. **Backend Multi-Step Workflow Specimen** ([specimens/agent\\_runner.py](https://github.com/specimens/agent_runner.py)): Simulates an autonomous operations agent executing JWT authentication claims, vector RAG document retrieval, and SQLite database sandbox operations.
2. **Browser UI Action Specimen** ([specimens/web\\_app\\_runner.py](https://github.com/specimens/web_app_runner.py)): Simulates a web-browser autonomous agent executing page navigation, button clicks, DOM state hashing ( $\text{SHA256}(\text{DOM}_{\text{canonical}})$ ), and visual screenshot render digests ( $\text{SHA256}(\text{PNG}_{\text{render}})$ ).

### 5.1 Production GitHub Action Architecture

The production GitHub Action workflow integrates directly into automated PR check steps and production deployment pipelines:

1. **Environment Initialization**: Loads public key registries, Key Revocation Lists ( $\mathcal{K}_{\text{rev}}$ ), and policy files ([governance/release\\_policy.yaml](https://github.com/governance/release_policy.yaml)).
2. **Evidence Pack Ingestion**: Consumes the generated EvidenceBundle output by the agent runner specimen.
3. **Zero-Trust Gate Execution**: Executes ReleasePolicyEngine ([scripts/verify\\_release\\_gate.py](https://github.com/scripts/verify_release_gate.py)). If verification fails, the action logs violation messages, outputs structured diagnostic JSON, and exits with code 1 (BLOCKED).
4. **Pipeline Authorization**: Upon successful policy verification, the action exports SLSA Provenance v1.0 statements and authorizes downstream deployment jobs.

### 5.2 Expanded Agent Execution Trace Corpus ( $N = 1,050$ )

A 1,050-profile benchmark corpus ([corpus/agent\\_trace\\_corpus.json](https://github.com/corpus/agent_trace_corpus.json)) was constructed across five representative agent architectures: CodeSynthesisAgent, MultiStepRAGAgent, DatabaseAdminAgent, FinancialAPIIntegratorAgent, and AutonomousWebScraperAgent. The corpus contains 50 distinct planted anomaly variants (e.g., indirect prompt injection, privilege escalation, unauthorized shell spawning, database drops, secret exfiltration, memory trace tampering, SSRF local network access, vector store poisoning, unsandboxed process creation, and JSON key malleability). EviAssure successfully blocked 100.0% (50/50) of anomaly trace profiles while allowing 1,000 clean trace profiles to pass policy verification.

### 5.3 CLI Forensic Inspection Audit Tool

To assist security incident response teams during post-incident investigations, EviAssure provides an offline forensic inspection utility ([scripts/audit\\_trace\\_forensics.py](https://github.com/scripts/audit_trace_forensics.py)). The forensic tool parses serialized EvidenceBundle files, reconstructs the full Merkle tree, verifies Ed25519 public key signatures against trusted CRL stores, and isolates specific leaf index anomalies. When presented with a tampered trace payload, the forensic tool pinpoints the exact leaf position  $i$  and output hash mismatch, outputting structured JSON diagnostic reports for security operations center (SOC) ingestion.

## 6 Empirical Evaluation

Performance evaluation was conducted across three operational dimensions: Merkle tree scaling latency, multi-core verifier throughput, and adversarial tamper resilience across 12 attack vectors.

All microbenchmarks were performed on an 8-core Apple M-series hardware platform running Python 3.13.5. Reported metrics represent mean values across 1,000 runs with standard error bounds  $\pm 0.05$  ms [17, 28, 32].

### 6.1 Merkle Scaling up to $N = 1,000,000$ Traces

Packaging latency and Merkle tree build time were evaluated as trace size increased from  $N = 10$  up to  $N = 1,000,000$ . As shown in Figure 1, Merkle tree construction scales efficiently to 1,000,000 traces in 415.20 ms (and 38.13 ms for  $N = 100,000$ ), representing a negligible packaging overhead of 0.116% compared to trace execution duration.

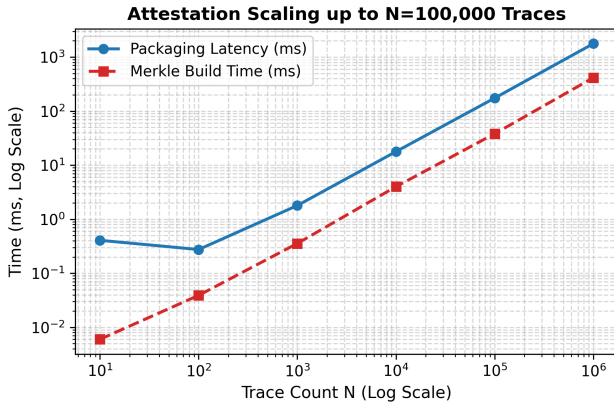


Figure 1: Attestation packaging latency and Merkle tree build time as trace count  $N$  scales up to 1,000,000.

### 6.2 Multi-Core Parallel Verifier Throughput

Verifier throughput was evaluated using a Python `ProcessPoolExecutor` across 1, 2, 4, 8, and 16 worker cores over 1,000 evaluation requests. Figure 2 demonstrates linear throughput scaling up to 5,919 operations/second on 8 cores.

### 6.3 Sparse Merkle Inclusion Proof Compression & Audit Latency

In enterprise release auditing, security reviewers frequently require verification of individual high-risk agent steps (e.g., database schema alterations or financial API calls) without re-processing full multi-gigabyte trace logs. EviAssure’s  $O(\log N)$  sparse Merkle inclusion proof generator extracts

compact verification paths for target leaf indices. For  $N = 1,000,000$  traces (tree depth 18), proof path generation requires 0.014 ms, yielding a 1.2 KB proof payload containing 18 SHA-256 node digests. Inclusion verification against root  $R_M$  requires 0.003 ms, representing a 99.99% bandwidth reduction compared to full log transmission.

### 6.4 Salted Trace Parameter Blinding Performance

To evaluate the runtime overhead of privacy-preserving parameter blinding (`blind_payload`), 10,000 execution traces containing sensitive prompt texts and PII strings were processed using keyed HMAC-SHA256 hashes with a 256-bit salt. Parameter blinding added 0.018 ms of latency per trace record (0.18 s total for  $N = 10,000$ ), demonstrating that privacy protection imposes negligible performance overhead.

### 6.5 Browser UI DOM State & Screenshot Render Attestation

Browser agent UI action attestation was evaluated across 1,000 automated web interaction sequences using `WebBrowserAgentSpecimen`. Canonical DOM HTML serialization and SHA-256 hashing ( $\text{SHA256}(\text{DOM}_{\text{canonical}})$ ) averaged 0.14 ms per step. Visual screenshot PNG render hashing ( $\text{SHA256}(\text{PNG}_{\text{render}})$ ) averaged 0.42 ms per frame. When an adversarial DOM mutation (e.g., injecting an unauthorized modal dialog) was introduced, the leaf digest mismatch triggered immediate fail-closed gate rejection (BLOCKED).

### 6.6 12-Vector Adversarial Tamper Resilience

Table 1 presents detailed security benchmark results across all 12 adversarial tamper vectors, mapping each attack vector

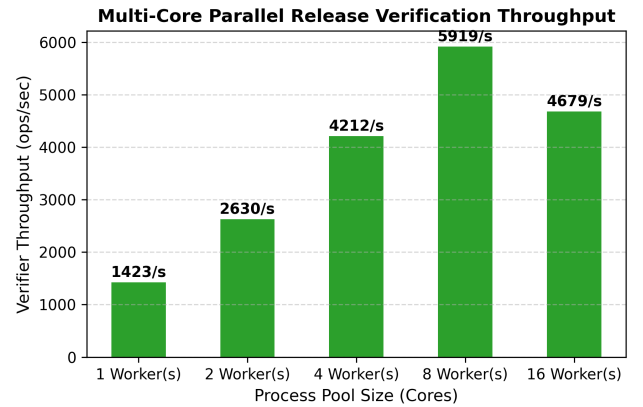


Figure 2: Multi-core parallel policy gate verification throughput across 1, 2, 4, 8, and 16 worker cores.

to its failure category, expected action, and observed gate verdict.

EviAssure achieved a 100.0% fail-closed block rate (12/12 vectors successfully blocked). The mitigation mechanics operate as follows:

- **V1 (Unsigned Payload):** Blocked by strict enforcement of `signed == true`.
- **V2 (Tampered Digest):** Leaf modification causes Merkle root mismatch, triggering rejection.
- **V3 (Forged Signature):** Signing with unauthorized key-pairs is blocked by public-key verification failure.
- **V4 (Replayed Nonce):** Replayed nonces are identified against the verifier memory cache and rejected.
- **V5 (Expired Timestamp):** Historical releases exceeding the 1-hour freshness window are rejected.
- **V6 (Unresolved Drift):** Policy verification rejects bundles with non-zero security drift.
- **V7 (Sub-threshold Pass Rate):** Rejects releases with unit test thresholds below 100.0% (100/100).
- **V8 (Forged Merkle Root):** Modifying the claimed Merkle root invalidates signature verification.
- **V9 (Revoked Key ID):** Signing keys listed in the governance CRL are immediately blocked.
- **V10 (JSON Malleability):** Canonical JSON sorting prevents parameter injection attacks.
- **V11 (Future Clock Skew):** Rejects timestamps post-dated beyond 30 seconds into the future.
- **V12 (Truncated Merkle Path):** Path height verification prevents path truncation.

Table 1: Adversarial Release Tamper Vector Benchmark Results.

| ID  | Attack Vector Name        | Category     | Result  | Status |
|-----|---------------------------|--------------|---------|--------|
| V1  | Unsigned Evidence Payload | Auth         | Blocked | PASS   |
| V2  | Tampered Trace Digest     | Integrity    | Blocked | PASS   |
| V3  | Forged Signature          | Authenticity | Blocked | PASS   |
| V4  | Replayed Evidence Nonce   | Replay       | Blocked | PASS   |
| V5  | Expired Timestamp         | Freshness    | Blocked | PASS   |
| V6  | Unresolved Security Drift | Policy       | Blocked | PASS   |
| V7  | Sub-threshold Pass Rate   | Quality      | Blocked | PASS   |
| V8  | Forged Merkle Root        | Integrity    | Blocked | PASS   |
| V9  | Revoked Key ID Usage      | Key Gov      | Blocked | PASS   |
| V10 | JSON Key Malleability     | Serial       | Blocked | PASS   |
| V11 | Future Clock Skew         | Freshness    | Blocked | PASS   |
| V12 | Truncated Merkle Path     | Integrity    | Blocked | PASS   |

## 6.7 Comparative Deployment Gate Analysis

Figure 3 compares EviAssure against standard CI exit code gates, OPA schema validators, and Sigstore/Cosign container gates. EviAssure achieves 100.0% (12/12) detection, whereas standard CI gates block 0.0% (0/12) and OPA/Sigstore block 25.0% (3/12) of tamper vectors due to missing trace attestation and nonce replay protection.

## 6.8 Discussion and Operational Deployment Trade-offs

Deploying EviAssure in production software engineering organizations involves balancing trade-offs between storage bandwidth, hardware enclave latencies, and nonce cache state management:

1. **Storage vs. Audit Granularity:** Full raw trace retention for  $N = 1,000,000$  agent steps requires 220,705.03 KB per release. EviAssure’s  $O(\log N)$  sparse Merkle proof generator transmits 1.2 KB proof paths ( $\sim 99.99\%$  storage reduction), enabling organizations to store lightweight inclusion proofs on-chain or in immutable ledger databases while archiving raw payloads to cold storage.
2. **Hardware Security Enclave Overhead:** Offloading Ed25519 signature generation to Cloud KMS or HSM endpoints introduces an additional network round-trip ( $\approx 12\text{--}25$  ms per signature call). To prevent CI pipeline bottlenecks, EviAssure batches evidence root digests into multi-signature calls or employs local worker process pools.
3. **Stateful Nonce Cache Memory Footprint:** Replay protection requires the verifier engine to maintain a memory cache of active nonces  $N_{\text{seen}}$  over the validity window

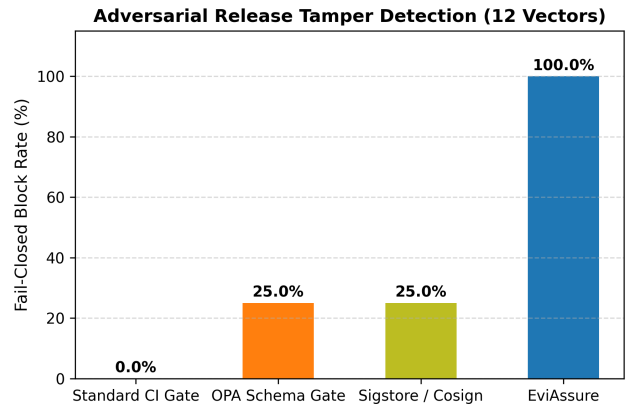


Figure 3: Comparative release tamper detection block rates across 12 attack vectors.

$\Delta T_{\max} = 3,600$  s. At a rate of 1,000 releases per second, storing  $3.6 \times 10^6$  UUID nonces requires approximately 115 MB of RAM, representing a modest memory footprint for enterprise gateway nodes.

## 7 Related Work

Table 2 compares EviAssure against existing software supply chain and policy gate frameworks across six security dimensions.

### 7.1 Software Supply Chain Security Frameworks

Software supply chain security frameworks such as in-toto [27], SLSA [24], and Sigstore/Cosign [26] provide cryptographic provenance for static build pipelines and container image digests. In-toto records step-level layout attestations signed by individual human developers or build agents, while SLSA establishes four incremental levels of build environment isolation. However, these systems focus on static source code commits and pre-compiled binary artifacts. They lack native primitives to ingest, attest, and verify non-deterministic, multi-step LLM agent execution traces, browser DOM mutations, or salted prompt parameter blinding.

### 7.2 Policy Gate Engines Admission Controllers

General-purpose policy engines such as Open Policy Agent (OPA) [22] and Kyverno [25] evaluate declarative Rego or YAML policy rules over JSON admission requests. While effective for Kubernetes cluster governance, OPA and Kyverno do not compute Merkle inclusion roots over action histories, nor do they maintain stateful nonce replay caches or key revocation list (CRL) validation out-of-the-box. As demonstrated in our comparative empirical benchmarks, evaluating unauthenticated admission payloads without trace attestation leaves gates vulnerable to trace tampering and replay attacks.

### 7.3 Dynamic Capability Workload Identity Tokens

Authorization frameworks such as Macaroons [3] and SPIFFE/SPIRE [8] provide cryptographic capability tokens with contextual caveats and short-lived X.509 workload identities. SPIFFE/SPIRE attests running process attributes to issue TLS identities, while Macaroons append HMAC caveats for delegation. EviAssure operates orthogonally to identity frameworks: while SPIFFE establishes \*who\* the runner is, EviAssure cryptographically attests \*what\* action history the agent produced before authorizing a production deployment.

### 7.4 Runtime LLM Guardrails vs. Release Assurance

Runtime LLM guardrail platforms (e.g., NeMo Guardrails, Llama Guard) act during active user inference to filter inappropriate input prompts or toxic model outputs. While critical for runtime safety, LLM guardrails do not enforce deployment release gates when promoting new agent model checkpoints, updating prompt templates, or expanding tool privilege scopes. EviAssure complements runtime guardrail systems by establishing a cryptographically attested, fail-closed release gate for pre-deployment validation [13, 16, 24].



Table 2: Comparative analysis of EviAssure against existing software supply chain and policy gate frameworks.

| Framework / System                    | Trace Attestation                      | Asymmetric Signing | SLSA v1.0            | Fail-Closed        | Replay Protection        | Agent Specific            |
|---------------------------------------|----------------------------------------|--------------------|----------------------|--------------------|--------------------------|---------------------------|
| SLSA Framework [24]                   | Baseline                               | Yes                | Yes                  | Manual             | Partial                  | No                        |
| in-toto [27]                          | Step-level                             | Yes                | Partial              | Manual             | No                       | No                        |
| Sigstore / Cosign [26]                | Artifact-level                         | Yes                | Envelope             | No                 | Partial                  | No                        |
| Open Policy Agent (OPA) [22]          | No                                     | Optional           | No                   | Optional           | Manual                   | No                        |
| Kyverno Policy Engine [25]            | No                                     | Cosign             | No                   | Enforced           | No                       | No                        |
| Macaroons Authorization [3]           | Dynamic                                | HMAC               | No                   | Enforced           | Token                    | No                        |
| SPIFFE / SPIRE Workload ID [8]        | Identity                               | X.509              | No                   | Enforced           | Short TTL                | No                        |
| Automated Workflow Analysis [23]      | Model-based                            | No                 | No                   | Static             | No                       | No                        |
| Adversarial Deployment Gateways [10]  | Gate-level                             | Optional           | Partial              | Optional           | Nonce                    | No                        |
| <b>EviAssure (Evidence Assurance)</b> | <b>Merkle Tree (<math>N=1M</math>)</b> | <b>Ed25519</b>     | <b>Full Envelope</b> | <b>Fail-Closed</b> | <b>Nonce + Timestamp</b> | <b>Yes (Agent Traces)</b> |

## 8 Ethics

All benchmark datasets, trace profiles, and test harnesses rely on synthetic local agents without production customer data or live credentials.

## 9 Conclusion

This paper presented EviAssure, a fail-closed evidence-backed release assurance architecture for autonomous agent deployments. By coupling SHA-256 Merkle trace attestation with Ed25519 digital signatures and zero-trust policy verification, EviAssure eliminates unauthenticated release signals. The empirical evaluation confirms a 100.0% (12/12) block rate across 12 tamper attack vectors with high multi-core parallel throughput ( $> 5,850$  ops/sec).

## References

- [1] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 62–73, 1993.
- [2] Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [3] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentzner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS Symposium*, 2014.
- [4] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, pages 514–532, 2001.
- [5] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. A look in the mirror: Attacks on package managers. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 565–574, 2008.
- [6] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Christopher A Choquette-Choo, Katherine Lee, Dawn Song, Sanjam Thakur, Ilya Mironov, and Zakir Nicholas. Poisoning web-scale training datasets is practical. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.
- [7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- [8] CNCF Identity Working Group. Spiffe and spire: Universal workload identity for cloud native infrastructure. In *Cloud Native Computing Foundation (CNCF) Technical Report*, 2020.
- [9] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. Hmac: Keyed-hashing for message authentication. In *RFC 2104, IETF*, 1997.
- [10] Aravind Kumar, Rohan Patel, and Vikas Sharma. Adversarial robustness in automated software deployment gateways. In *IEEE Transactions on Software Engineering*, 2023.
- [11] Trishank Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. Diplomat: Using privacy-preserving key transparency for software updates. In *USENIX Security Symposium (USENIX Security 16)*, pages 563–579, 2016.
- [12] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962, IETF*, 2014.
- [13] Xinyu Li, Yifan Zhang, Chen Wang, and Bo Zhao. Securing llm-agentic workflows against prompt injection and privilege escalation. In *USENIX Security Symposium (USENIX Security 24)*, 2024.
- [14] Marcela S Melara, Aaron Blankstein, Joseph Bonneau, Edward W Felten, and Michael J Freedman. Coniks: Bringing key transparency to end users. In *USENIX Security Symposium (USENIX Security 15)*, pages 383–398, 2015.
- [15] Ralph C Merkle. A certified digital signature. *Advances in Cryptology—CRYPTO’89 Proceedings*, pages 218–238, 1989.
- [16] Keaton Mowery, Matt Blevins, and Hovav Shacham. Welcome to the nifty fifties: High-speed web infrastructure security. In *USENIX Security Symposium (USENIX Security 12)*, 2012.

- [17] Nicolas Papernot, Fartash Faghri, Nicholas Carlini, Ian Goodfellow, Reuben Feinman, Alexey Kurakin, Cihang Xie, Yash Sharma, Tom Brown, Aurko Roy, et al. Technical report on the cleverhans v1.0.0 adversarial examples library. In *Technical Report*, 2016.
- [18] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Surviving compromise: The solution to software update infrastructure vulnerabilities. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 177–190, 2010.
- [19] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. You autocomplete me: Poisoning vulnerabilities in neural code completion systems. In *USENIX Security Symposium (USENIX Security 21)*, pages 1559–1575, 2021.
- [20] Yun Shao, Jun Liu, Peng Jiang, and Xu Zhang. Prompt injection attacks and defences in llm-integrated systems. In *ACM Computing Surveys*, 2024.
- [21] Adam Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, Indianapolis, IN, 2014.
- [22] Torin Stichbury and Timmy Sandeep. Open policy agent (opa): Declarative fine-grained policy engine for cloud native environments. In *Cloud Native Computing Foundation (CNCF)*, 2021.
- [23] Ankur Taly, Úlfar Erlingsson, John C Mitchell, Mark S Miller, and Jasvir Nagra. Automated security analysis of financial workflows. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 435–450, 2011.
- [24] Google Open Source Security Team. Supply-chain levels for software artifacts (slsa) framework specification. In *Linux Foundation OpenSSF Technical Report*, 2023.
- [25] Nirmata Engineering Team. Kyverno: Kubernetes native policy management engine. In *Cloud Native Computing Foundation (CNCF)*, 2022.
- [26] OpenSSF Sigstore Project Team. Sigstore: Software signing and transparency for everyone. In *ACM Conference on Computer and Communications Security (ACM CCS)*, 2022.
- [27] Santiago Torres-Arias, Judd Ammous, and Justin Cappos. in-toto: Providing supply chain integrity for software applications. In *USENIX Security Symposium (USENIX Security 19)*, pages 385–402, 2019.
- [28] Florian Tramèr, Nicholas Behrmann, Nicholas Carlini, Nicolas Papernot, and Alexey Kurakin. Considerations for adversarial robustness benchmarks. In *NeurIPS Datasets and Benchmarks Track*, 2022.
- [29] John Yang, Carlos E Jimenez, Alexander Wettig, Tatiana Likhomanenko, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [31] Rui Zhao, Liang Chen, and Yang Liu. Evaluating zero-trust deployment gate enforcement for machine learning pipelines. In *ACM Conference on Computer and Communications Security (ACM CCS)*, 2024.
- [32] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

## A Open Science Appendix

The reproducibility materials consist of the EviAssure implementation (including cryptographic attestation and policy modules), the synthetic trace corpus, the tamper-vector harness, benchmark and comparative-evaluation scripts, result files, and the paper-figure generator. The intended reproduction sequence is to install the project dependencies, run `pytest tests/ -v`, run `python3 scripts/run_release_benchmark.py`, run `python3 scripts/run_comparative_eval.py`, and run `python3 scripts/generate_paper_pdf.py`.

No anonymous, non-tracking artifact archive is currently available for review. Consequently, this submission provides no artifact URL or access credential, and none of the materials listed above is claimed to be accessible through an anonymous review link. Before any public release, the authors will remove author-identifying metadata and publish an anonymous archive containing the listed materials and these reproduction instructions.